

University of Padua

Gusatto Derek
2197482

Functional Languages
Homework Assignment

February 5, 2026

Contents

Instructions	2
List of Exercises	2
Solutions	3
Exercise 2.5.1	3
Exercise 2.5.2	4
Exercise 2.5.3	6
Exercise 3.4.4	8
Exercise 3.4.6	13
Exercise 4.3.1	17
Exercise 5.5.2	19
Exercise 5.5.3	21
Exercise Call-by-name equivalence	24

Instructions

For your final assignment, you are required to solve the following exercises.

- Solutions must be submitted as a PDF file generated using L^AT_EX.
- Cooperation is allowed, but each student must submit their own solution.
- There is no strict deadline; however, to have the final grade registered in a given exam round, the homework must be submitted at least two weeks before the round date.
- Solutions must be sent to francesco.gavazzo@unipd.it.

List of Exercises

All exercises refer to [Pitt's notes](#).

- Exercises 2.5.1, 2.5.2, 2.5.3
- Two exercises of your choice among 3.4.4, 3.4.5, 3.4.6, 3.4.7
- Exercise 4.3.1
- Exercises 5.5.2, 5.5.3, 5.5.4 (bonus exercise)

In Exercises 5.5.2–5.5.4, it is allowed (but not mandatory) to restrict the language to the pure λ -calculus, extended with integers and integer operations. All semantic statements of the form $\langle M, s \rangle R \langle M', s' \rangle$ must be replaced by $M R M'$, as done in class.

Solutions

Exercise 2.5.1. *Give an example of an SMC-machine configuration from which there is an infinite sequence of transitions.*

Solution. An example of an SMC-machine configuration that produces an infinite sequence of transitions is a **while** loop that never terminates.

Definition of the SMC Configuration A configuration of the SMC machine is defined as a triple $\langle c, r, \sigma \rangle$, so for a complete example, we must define:

- c (Control Stack): A stack of phrases (commands or expressions, as defined by the LC language) to be evaluated;
- r (Value Stack): A stack used for storing intermediate and final results;
- σ (Memory State): A partial function mapping a finite set of locations to integers, referred to as "state".

To create a never-ending loop we need an always true condition, like the equivalence between an integer and itself in the same state. To be sure that the state of the variable never changes we need the loop to execute an operation that doesn't change any variable state, e.g. the **skip** operation. We don't need any particular stack or memory, so we can use an empty stack (ε) and any state s . We obtain the following initial configuration:

$$\langle \mathbf{while\ 0 = 0\ do\ skip}, \varepsilon, s \rangle$$

Transition Logic and Divergence This specific configuration generates an infinite sequence of transitions for the following technical reasons:

- **Equivalence evaluation:** The Boolean expression $0 = 0$ is parsed by the SMC using its predefined binary operations. Since $n = n$ always evaluates to true, the loop condition will never change.
- **While loop semantics:** According to the informal semantics of the machine, the **while $n = n$ do skip** command is executed repeatedly as long as the expression $n = n$ remains true. The machine will parse the statement, evaluate for equality, execute the body (**skip**), and push the while command back onto the control stack.
- **Deterministic system:** The SMC is a deterministic transition system, meaning that the head of the control stack uniquely determines the next transition. Since there are no stopping conditions, the machine will continue to cycle through the states indefinitely.

□

Exercise 2.5.2. Consider the subset $\mathcal{D}$ of the set $\mathbb{N} \times \mathbb{N}$ of pairs of natural numbers defined by following axioms and rules

$$(n, 0) \in \mathcal{D}$$

$$\frac{(n, n') \in \mathcal{D}}{(n, n + n') \in \mathcal{D}}$$

Use Rule Induction to prove

$$(n, n') \in \mathcal{D} \implies \exists k \in \mathbb{N}. n' = k * n$$

(where $*$ denotes multiplication). Use Mathematical Induction on k to show conversely that if $n' = k * n$ then $(n, n') \in \mathcal{D}$

Solution.

1. **Rule induction** We claim

$$(n, n') \in \mathcal{D} \implies \exists k \in \mathbb{N}. n' = k * n$$

We prove this using rule induction, which follows the way the set $\mathcal{D}$ is defined.

1.1 **BASE CASE** The base rule says:

$$(n, 0) \in \mathcal{D}$$

We need to show there exists $k \in \mathbb{N}$ such that $n' = k * n = 0$.

Choose $k = 0$. Then clearly:

$$0 * n = 0$$

Thus the base case holds.

1.2 **INDUCTIVE STEP** The inductive rule says:

$$\frac{(n, n') \in \mathcal{D}}{(n, n + n') \in \mathcal{D}}$$

Assume as the induction hypothesis:

$$(n, n') \in \mathcal{D} \text{ and } n' = kn \implies n + n' = (k + 1) * n$$

We must show:

$$(n, n + n') \in \mathcal{D} \implies \exists k \in \mathbb{N}. n + n' = k * n$$

By induction hypothesis there exists some $k \in \mathbb{N}$ such that $n' = k * n$. Then:

$$n + n' = n + k * n = (k + 1) * n$$

Let $k' = k + 1$. Then $n + n' = k' * n$, with $k' \in \mathbb{N}$.

Thus, the inductive step holds.

By rule induction we can conclude

$$(n, n') \in \mathcal{D} \implies \exists k \in \mathbb{N}. n' = k * n$$

2. **Mathematical Induction** Now we prove the converse. Claim:

$$\forall n \in \mathbb{N}, \forall k \in \mathbb{N}, (n, k * n) \in \mathcal{D}$$

We will use mathematical induction on k .

2.1 BASE CASE ($k = 0$) We need to show:

$$(n, 0 * n) = (n, 0) \in \mathcal{D}$$

This is exactly the base rule of $\mathcal{D}$, thus the base case holds.

2.2 INDUCTIVE STEP Assume the inductive hypothesis for some $k \in \mathbb{N}$:

$$(n, k * n) \in \mathcal{D}$$

We need to prove:

$$(n, (k + 1) * n) \in \mathcal{D}$$

Observe that

$$(k + 1) * n = k * n + n$$

By inductive hypothesis $(n, k * n) \in \mathcal{D}$ and by inductive rule of $\mathcal{D}$, adding n to the second component gives:

$$(n, k * n + n) = (n, (k + 1) * n) \in \mathcal{D}$$

Thus the inductive step holds.

By mathematical induction on k we conclude:

$$(n, k * n) \in \mathcal{D} \quad \forall k \in \mathbb{N}, \forall n \in \mathbb{N}$$

We conclude that $\mathcal{D}$ is exactly the set of pairs (n, n') such that $n' = k * n$ for some $k \in \mathbb{N}$:

$$\mathcal{D} = \{(n, k * n) \mid n \in \mathbb{N}, k \in \mathbb{N}\}$$

□

Exercise 2.5.3. Let $(Config, \rightarrow)$ be a transition system. Give an inductive definition of the subset of $Config \times Config$ consisting of the reflexive-transitive closure of $\rightarrow$. Use Mathematical Induction and Rule Induction to prove that your definition gives the same relation as

(Definition 1.1.1(i)) $\rightarrow^*$ denotes the binary relation on $Config$ which is the reflexive-transitive closure of $\rightarrow$. In other words $c \rightarrow^* c'$ holds just in

$$c = c_0 \rightarrow c_1 \rightarrow \cdots \rightarrow c_{n+1} \rightarrow c_n = c'$$

holds for some $c_0, \dots, c_n \in Config$ (where $n \geq 0$; the case $n = 0$ just means $c = c'$).

Solution.

1. **Inductive Definition of Reflexive-Transitive Closure** Let $(Config, \rightarrow)$ be a transition system, where $Config$ is the set of configurations and $\rightarrow$ is the transition relation. The iterated transition relation $\rightarrow^*$ is defined inductively by the following:

- Reflexivity axiom:

$$\frac{}{s \rightarrow^* s}$$

- Transition step rule:

$$\frac{s \rightarrow s' \quad s' \rightarrow^* s''}{s \rightarrow^* s''}$$

2. **Equivalence Proof** We need to show that the two definitions generate the same relation, i.e. the inductive relation (R_{IND}) is equal to the relation given by *Definition 1.1.1(i)* (R_{DEF}).

2.1 $R_{IND} \subseteq R_{DEF}$.

To show that every pair (s, s') defined by the inductive rules satisfies *Definition 1.1.1(i)* we use rule induction.

2.1.1 CLOSURE UNDER AXIOM. We must show that $s \rightarrow^* s$ according to *Definition 1.1.1(i)*. This is true for $n = 0$, where $s = s_0 = s'$

2.1.2 CLOSURE UNDER TRANSITION RULE. Assume that $s \rightarrow^* s'$ and that the inductive hypothesis $s' \rightarrow^* s''$ holds (i.e., there exists a sequence of length n from s' to s''). Then by *Definition 1.1.1(i)* we can build a sequence of length $n + 1$ adding $s \rightarrow s'$ before the existing sequence, thus $s \rightarrow^* s''$ is satisfied.

2.2 $R_{DEF} \subseteq R_{IND}$

To show that every sequence of transitions in *Definition 1.1.1(i)* is derivable from the inductive rules, we proceed by Mathematical Induction on the number of steps n of the sequence.

2.2.1 BASE CASE ($n = 0$) If the sequence has 0 steps, then $s = s'$. By the reflexivity axiom we know that $s \rightarrow^* s'$ is always derivable.

2.2.2 INDUCTIVE STEP Assume that for all sequences of length n from s to s_{n+1} $s \rightarrow^* s_{n+1}$ in the inductive system. Consider a sequence of length $n + 1$:

$$s \rightarrow s_1 \rightarrow \cdots \rightarrow s_{n+1}$$

2.2.2.1 From the sequence, we know that there is a direct step $s' \rightarrow s_1$.

2.2.2.2 The remainder of the sequence $(s_1 \rightarrow \cdots \rightarrow s_{n+1})$ has length n . By the inductive hypothesis, this sequence implies that $(s_1 \rightarrow^* s_{n+1})$ is inductively derivable.

2.2.2.3 Applying the transition step rule on $s \rightarrow s_1$ and $s_1 \rightarrow^* s_{n+1}$ we obtain the derivation of $s' \rightarrow^* s_{n+1}$.

3. **Conclusion** Since we have proved inclusion in both directions, the inductive definition and *Definition 1.1.1(i)* describe the same reflexive-transitive closure relation.

□

Exercise 3.4.4. Call LC Configuration $\langle P, s \rangle$ *sensible* if the set of locations on which s is defines, $\text{dom}(s)$, contains all the locations that occur in the phrase P . Prove by induction on the structure of P that if $\langle P, s \rangle$ is *sensible*, then it is not *stuck*. Prove by Rule Induction for $\rightarrow$ that if $\langle P, s \rangle \rightarrow \langle P', s' \rangle$ and $\langle P, s \rangle$ is *sensible*, then so is $\langle P', s' \rangle$ and $\text{dom}(s') = \text{dom}(s)$. Deduce that a *stuck* configuration can never be reached by a series of transitions from a *sensible* configuration.

Solution.

1. **Definitions.** We define:

- $\langle P, s \rangle$ *sensible* if the set of locations on which s is defines, $\text{dom}(s)$, contains all the locations that occur in the phrase P ;
- $\langle P, s \rangle$ *stuck* if it is not terminal and there is no possible transition $\langle P, s \rangle \rightarrow \langle P', s' \rangle$.

2. **By structural induction, *sensible* $\implies$ not *stuck*.**

We want to show that if $\langle P, s \rangle$ is *sensible*, then it is not *stuck*. We proceed by structural induction on P .

2.1 BASE CASES

2.1.1 **Terminal configurations** ($P = n$ (integer), $P = b$ (boolean value) or $P = \text{skip}$). By definition P is terminal and a *stuck* configuration is by definition not terminal, thus P is not *stuck*;

2.1.2 **Memory lookup** ($P = !\ell$). If $\langle !\ell, s \rangle$ is *sensible* then $\ell \in \text{dom}(s)$ hence by

$$(\vec{loc}) \quad \frac{}{\langle !\ell, s \rangle \rightarrow \langle n, s \rangle} \text{ with } s(\ell) = n$$

there is a transition. Since a such a configuration exists, $\langle !\ell, s \rangle$ is not *stuck*.

2.2 INDUCTIVE STEP. Assume by inductive hypothesis (IH) that the property we're studying holds for P_1 and P_2 .

2.2.1 **Binary operations** ($P = P_1 \text{ op } P_2$)

2.2.1.1 if P_1 is not a value, then by IH there is a configuration $\langle P_1, s \rangle \rightarrow \langle P'_1, s' \rangle$.
By the rule

$$(\vec{op}_1) \quad \frac{\langle E_1, s \rangle \rightarrow \langle E'_1, s' \rangle}{\langle E_1 \text{ op } E_2, s \rangle \rightarrow \langle E'_1 \text{ op } E_2, s' \rangle}$$

there is a configuration for P , so it is not *stuck*;

2.2.1.2 if P_1 is a value, but P_2 isn't the by IH there is a configuration $\langle P_2, s \rangle \rightarrow \langle P'_2, s' \rangle$. By the rule

$$(\vec{op}_2) \quad \frac{\langle E_2, s \rangle \rightarrow \langle E'_2, s' \rangle}{\langle n \text{ op } E_2, s \rangle \rightarrow \langle n \text{ op } E'_2, s' \rangle}$$

there is a configuration for P , so it is not *stuck*;

2.2.1.3 if both P_1 and P_2 are values, by the rule

$$(\vec{op}_3) \quad \frac{}{\langle n \text{ op } m, s \rangle \rightarrow \langle c, s \rangle} \text{ with } c = n \text{ op } m$$

there is a configuration for P , so it is not *stuck*;

2.2.2 **Assignment** ($P = \ell := P_1$)

2.2.2.1 if P_1 is not a value, then by IH there is a configuration $\langle P_1, s \rangle \rightarrow \langle P'_1, s' \rangle$.

By the rule

$$(\vec{set}_1) \frac{\langle E, s \rangle \rightarrow \langle E', s' \rangle}{\langle \ell := E, s \rangle \rightarrow \langle \ell := E', s' \rangle}$$

there is a configuration for P , so it is not *stuck*;

2.2.2.2 if P_1 is a value n since $\ell \in \text{dom}(s)$ we can use the rule

$$(\vec{set}_2) \frac{}{\langle \ell := n, s \rangle \rightarrow \langle \text{skip}, s[\ell \mapsto n] \rangle}$$

without the risk of a *stuck* configuration, then there is a configuration for P , so it is not *stuck*;

2.2.3 Composition ($P = P_1; P_2$)

2.2.3.1 if P_1 is not a skip, then by IH there is a configuration $\langle P_1, s \rangle \rightarrow \langle P'_1, s' \rangle$.

By the rule

$$(\vec{seq}_1) \frac{\langle C_1, s \rangle \rightarrow \langle C'_1, s' \rangle}{\langle C_1; C_2, s \rangle \rightarrow \langle C'_1; C_2, s' \rangle}$$

there is a configuration for P , so it is not *stuck*;

2.2.3.2 if P_1 is a value, then by the rule

$$(\vec{seq}_2) \frac{}{\langle \text{skip}; C_2, s \rangle \rightarrow \langle C_2, s \rangle}$$

there is a configuration for P , so it is not *stuck*;

2.2.4 If-Then-Else ($P = \text{if } B \text{ then } C_1 \text{ else } C_2$) We assume $\langle P, s \rangle = \langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle$ to be sensible, i.e. $\text{Loc}(B) \cup \text{Loc}(C_1) \cup \text{Loc}(C_2) \subseteq \text{dom}(s)$, so in particular

- $\langle B, s \rangle$ is sensible;
- $\langle C_1, s \rangle$ is sensible;
- $\langle C_2, s \rangle$ is sensible.

We can have three cases on the boolean expression B :

2.2.4.1 B is not yet a value, then we can apply

$$(\vec{if}_1) \frac{\langle B, s \rangle \rightarrow \langle B', s' \rangle}{\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle \text{if } B' \text{ then } C_1 \text{ else } C_2, s' \rangle}$$

2.2.4.2 $B = \text{true}$, then we can apply

$$(\vec{if}_2) \frac{}{\langle \text{if } \text{true} \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle C_1, s \rangle}$$

2.2.4.3 $B = \text{false}$, then we can apply

$$(\vec{if}_3) \frac{}{\langle \text{if } \text{false} \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle C_2, s \rangle}$$

Hence in all cases a transition exists, therefore $\langle P, s \rangle$ is not stuck.

2.2.5 While ($P = \text{while } B \text{ do } C$) We assume $\langle P, s \rangle = \langle \text{while } B \text{ do } C, s \rangle$ to be sensible, i.e. $\text{Loc}(B) \cup \text{Loc}(C) \subseteq \text{dom}(s)$. A while loop is never evaluated directly, instead it unfolds into an if statement:

$$(\vec{whl}) \frac{}{\langle \text{while } B \text{ do } C, s \rangle \rightarrow \langle \text{if } B \text{ then } (C; \text{while } B \text{ do } C) \text{ else } \text{skip}, s \rangle}$$

So a transition always exists, thus $\langle P, s \rangle$ is not stuck.

3. By rule induction, preservation of *sensible* configuration.

We want to prove that if $\langle P, s \rangle \rightarrow \langle P', s' \rangle$ and $\langle P, s \rangle$ is *sensible*, then so is $\langle P', s' \rangle$ and $\text{dom}(s') = \text{dom}(s)$.

3.1 CLOSURE UNDER AXIOMS

3.1.1 Memory lookup

$$(\vec{loc}) \quad \frac{}{\langle !\ell, s \rangle \rightarrow \langle n, s \rangle} \text{ with } s(\ell) = n$$

Note that the memory state doesn't change $s' = s$, so $\text{dom}(s') = \text{dom}(s)$. Then $s(\ell)$ is an integer which doesn't contain locations ($\text{Loc}(s(\ell)) = \emptyset$) and the empty set is a subset of every domain, hence the resulting configuration is *sensible*.

3.1.2 Assignment

$$(\vec{set}_2) \quad \frac{}{\langle \ell := n, s \rangle \rightarrow \langle \text{skip}, s[\ell \mapsto n] \rangle}$$

Since the starting configuration is *sensible* $\ell \in \text{dom}(s)$, hence $s[\ell \mapsto n]$ changes the value of an existing location without changing the set of locations, so $\text{dom}(s') = \text{dom}(s)$.

3.1.3 Integer operation

$$(\vec{op}_3) \quad \frac{}{\langle n \text{ op } m, s \rangle \rightarrow \langle c, s \rangle} \text{ with } c = n \text{ op } m$$

Note that the memory state doesn't change $s' = s$, so $\text{dom}(s') = \text{dom}(s)$. Then c is an integer which doesn't contain locations ($\text{Loc}(c) = \emptyset$) and the empty set is a subset of every domain, hence the resulting configuration is *sensible*.

3.1.4 Skip

$$(\vec{seq}_2) \quad \frac{}{\langle \text{skip}; C_2, s \rangle \rightarrow \langle C_2, s \rangle}$$

By hypothesis $\langle \text{skip}; C_2, s \rangle$ is *sensible*, i.e. $\text{Loc}(\text{skip}) = \emptyset \cup \text{Loc}(C_2) \subseteq \text{dom}(s)$. Since the memory state doesn't change $s' = s$, so $\text{dom}(s') = \text{dom}(s)$, hence $\text{Loc}(C_2) \subseteq \text{dom}(s')$ so the resulting configuration is *sensible*.

3.1.5 If-True

$$(\vec{if}_2) \quad \frac{}{\langle \text{if } \text{true} \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle C_1, s \rangle}$$

By hypothesis $\langle \text{if } \text{true} \text{ then } C_1 \text{ else } C_2, s \rangle$ is *sensible*, i.e. $\text{Loc}(B) \cup \text{Loc}(C_1) \cup \text{Loc}(C_2) \subseteq \text{dom}(s)$. Since the memory state doesn't change $s' = s$, so $\text{dom}(s') = \text{dom}(s)$, hence $\text{Loc}(B) \cup \text{Loc}(C_1) \cup \text{Loc}(C_2) \subseteq \text{dom}(s')$ so the resulting configuration is *sensible*.

3.1.6 If-False

$$(\vec{if}_3) \quad \frac{}{\langle \text{if } \text{false} \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle C_2, s \rangle}$$

By hypothesis $\langle \text{if } \text{false} \text{ then } C_1 \text{ else } C_2, s \rangle$ is *sensible*, i.e. $\text{Loc}(B) \cup \text{Loc}(C_1) \cup \text{Loc}(C_2) \subseteq \text{dom}(s)$. Since the memory state doesn't change $s' = s$, so $\text{dom}(s') = \text{dom}(s)$, hence $\text{Loc}(B) \cup \text{Loc}(C_1) \cup \text{Loc}(C_2) \subseteq \text{dom}(s')$ so the resulting configuration is *sensible*.

3.1.7 While

$$(\vec{whl}) \quad \frac{}{\langle \text{while } B \text{ do } C, s \rangle \rightarrow \langle \text{if } B \text{ then } (\text{while } B \text{ do } C) \text{ else } \text{skip}, s \rangle}$$

By hypothesis $\langle \text{while } B \text{ do } C, s \rangle$ is *sensible*, i.e. $\text{Loc}(B) \cup \text{Loc}(C) \subseteq \text{dom}(s)$. Since the memory state doesn't change $s' = s$, so $\text{dom}(s') = \text{dom}(s)$, hence $\text{Loc}(B) \cup \text{Loc}(C) \subseteq \text{dom}(s')$ so the resulting configuration is *sensible*.

3.2 CLOSURE UNDER RULES

3.2.1 Binary operation

$$(\vec{op}_1) \quad \frac{\langle E_1, s \rangle \rightarrow \langle E'_1, s' \rangle}{\langle E_1 \text{ op } E_2, s \rangle \rightarrow \langle E'_1 \text{ op } E_2, s' \rangle}$$

Assume $\langle E_1 \text{ op } E_2, s \rangle$ *sensible*, i.e. $Loc(E_1) \cup Loc(E_2) \subseteq dom(s)$.

By Inductive Hypothesis (IH) for $\langle E_1, s \rangle \rightarrow \langle E'_1, s' \rangle$ we have that $\langle E'_1, s' \rangle$ is *sensible*, i.e. $Loc(E'_1) \subseteq dom(s')$ and $dom(s') = dom(s)$.

Since $Loc(E_2) \subseteq dom(s)$ and $dom(s') = dom(s)$, $Loc(E_2) \subseteq dom(s')$.

Hence $Loc(E_2) \subseteq dom(s')$ and $Loc(E'_1) \subseteq dom(s')$ thus $Loc(E'_1) \cup Loc(E_2) \subseteq dom(s')$ so the resulting configuration is *sensible*.

3.2.2 Assignment

$$(\vec{set}_1) \quad \frac{\langle E, s \rangle \rightarrow \langle E', s' \rangle}{\langle \ell := E, s \rangle \rightarrow \langle \ell := E', s' \rangle}$$

Assume $\langle \ell := E, s \rangle$ is *sensible*, i.e. $Loc(\ell := E) = \{\ell\} \cup Loc(E) \subseteq dom(s)$.

By Inductive Hypothesis (IH) for $\langle E, s \rangle \rightarrow \langle E', s' \rangle$ we have that $\langle E', s' \rangle$ is *sensible*, i.e. $Loc(E') \subseteq dom(s')$ and $dom(s') = dom(s)$.

Since $Loc(\ell) \subseteq dom(s)$ and $dom(s') = dom(s)$, $Loc(\ell) \subseteq dom(s')$.

Hence $Loc(\ell) \subseteq dom(s')$ and $Loc(E') \subseteq dom(s')$ thus $Loc(\ell) \cup Loc(E') \subseteq dom(s')$ so the resulting configuration is *sensible*.

3.2.3 Sequence

$$(\vec{seq}_1) \quad \frac{\langle C_1, s \rangle \rightarrow \langle C'_1, s' \rangle}{\langle C_1; C_2, s \rangle \rightarrow \langle C'_1; C_2, s' \rangle}$$

Assume $\langle C_1; C_2, s \rangle$ *sensible*, i.e. $Loc(C_1) \cup Loc(C_2) \subseteq dom(s)$.

By Inductive Hypothesis (IH) for $\langle C_1, s \rangle \rightarrow \langle C'_1, s' \rangle$ we have that $\langle C'_1, s' \rangle$ is *sensible*, i.e. $Loc(C'_1) \subseteq dom(s')$ and $dom(s') = dom(s)$.

Since $Loc(C_2) \subseteq dom(s)$ and $dom(s') = dom(s)$, $Loc(C_2) \subseteq dom(s')$.

Hence $Loc(C_2) \subseteq dom(s')$ and $Loc(C'_1) \subseteq dom(s')$ thus $Loc(C'_1) \cup Loc(C_2) \subseteq dom(s')$ so the resulting configuration is *sensible*.

3.2.4 If-Then-else

$$(\vec{if}_1) \quad \frac{\langle B, s \rangle \rightarrow \langle B', s' \rangle}{\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle \text{if } B' \text{ then } C_1 \text{ else } C_2, s' \rangle}$$

Assume $\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle$ *sensible*, i.e. $Loc(B) \cup Loc(C_1) \cup Loc(C_2) \subseteq dom(s)$.

By Inductive Hypothesis (IH) for $\langle B, s \rangle \rightarrow \langle B', s' \rangle$ we have that $\langle B', s' \rangle$ is *sensible*, i.e. $Loc(B') \subseteq dom(s')$ and $dom(s') = dom(s)$.

Since $Loc(C_1) \cup Loc(C_2) \subseteq dom(s)$ and $dom(s') = dom(s)$, $Loc(C_1) \cup Loc(C_2) \subseteq dom(s')$.

Hence $Loc(C_1) \cup Loc(C_2) \subseteq dom(s')$ and $Loc(B') \subseteq dom(s')$ thus $Loc(C'_1) \cup Loc(C_2) \cup Loc(B') \subseteq dom(s') = dom(s)$ so the resulting configuration is *sensible*.

4. A *stuck* configuration can never be reached from a *sensible* configuration.

In fact we've proven that a transition from a *sensible* configuration brings a another *sensible* configuration (3). We can prove by induction on the number of steps n we can extend this property to every finite computation $\langle P, s \rangle \rightarrow^* \langle P_n, s_n \rangle$ (reflexive-transitive closure of $\rightarrow$) to show that every reachable configuration starting from a *sensible* configuration is *sensible* too.

4.1 BASE CASE ($n = 0$) It's the same configuration.

4.2 INDUCTIVE CASE ($n = k + 1$) We know by Inductive Hypothesis that if $\langle P, s \rangle$ is *sensible*, then $\langle P, s \rangle \rightarrow^* \langle P_k, s_k \rangle$ brings to $\langle P_k, s_k \rangle$ *sensible* and by point 2 we know that it's not stuck. Then there is $\langle P_k, s_k \rangle \rightarrow \langle P_{k+1}, s_{k+1} \rangle$ which by point 3 it is *sensible*.

We know that every *sensible* configuration is not *stuck* (2) hence every configuration reachable from a *sensible* configuration is not *stuck*.

□

Exercise 3.4.6. Complete the details of the proofs of properties (a) and (b).
(Equivalence of LC transition and evaluation semantics) For all configurations $\langle P, s \rangle$ and all terminal configuration $\langle V, s' \rangle$

$$\langle P, s \rangle \Downarrow \langle V, s' \rangle \iff \langle P, s \rangle \rightarrow^* \langle V, s' \rangle$$

$$(a) \quad \langle P, s \rangle \Downarrow \langle V, s' \rangle \implies \langle P, s \rangle \rightarrow^* \langle V, s' \rangle$$

$$(b) \quad \langle P, s \rangle \rightarrow \langle P', s' \rangle \& \langle P', s' \rangle \Downarrow \langle V, s'' \rangle \implies \langle P, s \rangle \Downarrow \langle V, s'' \rangle$$

Solution.

$$(a) \quad \langle P, s \rangle \Downarrow \langle V, s' \rangle \implies \langle P, s \rangle \rightarrow^* \langle V, s' \rangle$$

By Rule Induction, to prove (a) it suffices to show that the property is closed under the axioms and rules inductively defining .

(a).1 **Values (integer, boolean) and skip.** In this cases the only applicable rule is

$$(\Downarrow_{con}) \quad \langle c, s \rangle \Downarrow \langle c, s \rangle \text{ with } c \in \mathbb{Z} \cup \mathbb{B} \cup \{skip\}$$

By reflexive-transitive closure of $\rightarrow$,

$$\langle V, s \rangle \rightarrow^* \langle V, s \rangle$$

holds for all V .

(a).2 **Memory lookup**

$$(\Downarrow_{loc}) \quad \langle !\ell, s \rangle \Downarrow \langle n, s \rangle \text{ with } \ell \in \text{dom}(s) \text{ and } s(\ell) = n$$

We can use the axiom

$$\langle !\ell, s \rangle \rightarrow \langle n, s \rangle \text{ with } \ell \in \text{dom}(s) \text{ and } s(\ell) = n$$

By reflexive-transitive closure of $\rightarrow$ then

$$\langle !\ell, s \rangle \rightarrow^* \langle n, s \rangle \text{ with } \ell \in \text{dom}(s) \text{ and } s(\ell) = n$$

(a).3 **Binary operations**

$$(\Downarrow_{op}) \quad \frac{\langle E_1, s \rangle \Downarrow \langle n_1, s' \rangle \quad \langle E_2, s' \rangle \Downarrow \langle n_2, s'' \rangle}{\langle E_1 \text{ op } E_2, s \rangle \Downarrow \langle c, s'' \rangle} \text{ with } c = n_1 \text{ op } n_2$$

By Inductive Hypothesis (IH) we have that

$$(i) \quad \langle E_1, s \rangle \rightarrow^* \langle n_1, s' \rangle;$$

$$(ii) \quad \langle E_2, s' \rangle \rightarrow^* \langle n_2, s'' \rangle;$$

and we have the axiom

$$(\vec{op}_3) \quad \frac{}{\langle n \text{ op } m, s \rangle \rightarrow \langle c, s \rangle} \text{ with } c = n \text{ op } m$$

Hence

$$\langle E_1 \text{ op } E_2, s \rangle \xrightarrow{(i)}^* \langle n_1 \text{ op } E_2, s' \rangle \xrightarrow{(ii)}^* \langle n_1 \text{ op } n_2, s'' \rangle \xrightarrow{(\vec{op}_3)} \langle c, s'' \rangle$$

which can be rewritten as

$$\langle E_1 \text{ op } E_2, s \rangle \rightarrow^* \langle c, s'' \rangle$$

(a).4 **Assignment**

$$(\Downarrow_{set}) \quad \frac{\langle E, s \rangle \Downarrow \langle n, s' \rangle}{\langle \ell := E, s \rangle \Downarrow \langle skip, s'[\ell \mapsto n] \rangle}$$

By Inductive Hypothesis (IH) we have that $\langle E, s \rangle \rightarrow^* \langle n, s' \rangle$. By axiom

$$(\overrightarrow{set_2}) \quad \frac{}{\langle \ell := n, s \rangle \rightarrow \langle skip, s[\ell \mapsto n] \rangle}$$

and by reflexive-transitive closure of $\rightarrow$, it follows that

$$\langle \ell := E, s \rangle \rightarrow^* \langle \ell := n, s' \rangle \rightarrow^* \langle skip, s'[\ell \mapsto n] \rangle$$

(a).5 **Sequence**

$$(\Downarrow_{seq}) \quad \frac{\langle C_1, s \rangle \Downarrow \langle skip, s' \rangle \quad \langle C_2, s' \rangle \Downarrow \langle skip, s'' \rangle}{\langle C_1; C_2, s \rangle \Downarrow \langle skip, s'' \rangle}$$

By Inductive Hypothesis (IH) we have that

- (i) $\langle C_1, s \rangle \rightarrow^* \langle skip, s' \rangle$;
- (ii) $\langle C_2, s' \rangle \rightarrow^* \langle skip, s'' \rangle$;

and we have the axiom

$$(\overrightarrow{seq_2}) \quad \frac{}{\langle skip; C_2, s \rangle \rightarrow \langle C_2, s \rangle}$$

and the rule

$$(\overrightarrow{seq_1}) \quad \frac{\langle C_1, s \rangle \rightarrow \langle C'_1, s' \rangle}{\langle C_1; C_2, s \rangle \rightarrow \langle C'_1; C_2, s' \rangle}$$

Hence

$$\langle C_1; C_2, s \rangle \xrightarrow{(\overrightarrow{seq_2}) \& (i)}^* \langle skip; C_2, s' \rangle \xrightarrow{(\overrightarrow{seq_1}) \& (ii)}^* \langle skip, s'' \rangle$$

(a).6 **If-true**

$$(\Downarrow_{if1}) \quad \frac{\langle B, s \rangle \Downarrow \langle true, s' \rangle \quad \langle C_1, s' \rangle \Downarrow \langle skip, s'' \rangle}{\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle \Downarrow \langle skip, s'' \rangle}$$

By Inductive Hypothesis (IH) we have that

- (i) $\langle B, s \rangle \rightarrow^* \langle true, s' \rangle$;
- (ii) $\langle C_1, s' \rangle \rightarrow^* \langle skip, s'' \rangle$;

and we have the axiom

$$(\overrightarrow{if2}) \quad \frac{}{\langle \text{if } true \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle C_1, s \rangle}$$

Hence

$$\begin{aligned} \langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle &\xrightarrow{(i)}^* \langle \text{if } true \text{ then } C_1 \text{ else } C_2, s' \rangle \xrightarrow{(\overrightarrow{if2})}^* \\ &\xrightarrow{(i)}^* \langle C_1, s' \rangle \rightarrow^* \langle skip, s'' \rangle \end{aligned}$$

(a).7 **If-false**

$$(\Downarrow_{if2}) \quad \frac{\langle B, s \rangle \Downarrow \langle false, s' \rangle \quad \langle C_2, s' \rangle \Downarrow \langle skip, s'' \rangle}{\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle \Downarrow \langle skip, s'' \rangle}$$

By Inductive Hypothesis (IH) we have that

- (i) $\langle B, s \rangle \rightarrow^* \langle false, s' \rangle$;
- (ii) $\langle C_2, s' \rangle \rightarrow^* \langle skip, s'' \rangle$;

and we have the axiom

$$(if3) \quad \frac{\vec{}}{} \overline{\langle if \ false \ then \ C_1 \ else \ C_2, s \rangle \rightarrow \langle C_2, s \rangle}$$

Hence

$$\begin{aligned} \langle if \ B \ then \ C_1 \ else \ C_2, s \rangle &\xrightarrow{(i)} \langle if \ false \ then \ C_1 \ else \ C_2, s' \rangle \xrightarrow{(if3)} \\ &\langle C_2, s' \rangle \xrightarrow{(i)} \langle skip, s'' \rangle \end{aligned}$$

(a).8 **While-false**

$$(\Downarrow_{wh2}) \quad \frac{\langle B, s \rangle \Downarrow \langle false, s' \rangle}{\langle while \ B \ do \ C, s \rangle \Downarrow \langle skip, s' \rangle}$$

By Inductive Hypothesis (IH) we have that

- (i) $\langle B, s \rangle \rightarrow^* \langle false, s' \rangle$;

and we have the rule

$$(\vec{whl}) \quad \overline{\langle while \ B \ do \ C, s \rangle \rightarrow \langle if \ B \ then \ (C; while \ B \ do \ C) \ else \ skip, s \rangle}$$

Hence

$$\begin{aligned} \langle while \ B \ do \ C, s \rangle &\xrightarrow{(\vec{whl})} \langle if \ B \ then \ (C; while \ B \ do \ C) \ else \ skip, s \rangle \xrightarrow{(i)} \\ &\langle if \ false \ then \ (C; while \ B \ do \ C) \ else \ skip, s' \rangle \xrightarrow{(if3)} \langle skip, s' \rangle \end{aligned}$$

- (b) $\langle P, s \rangle \rightarrow \langle P', s' \rangle \& \langle P', s' \rangle \Downarrow \langle V, s'' \rangle \implies \langle P, s \rangle \Downarrow \langle V, s'' \rangle$

By Rule Induction, to prove (b) it suffices to show that the property is closed under the axioms and rules inductively defining .

(b).1 **Memory lookup**

$$\langle !\ell, s \rangle \rightarrow \langle n, s \rangle \text{ with } s(\ell) = n$$

Assume $\langle n, s \rangle \Downarrow \langle V, s'' \rangle$. Since n is an integer final value, by $\Downarrow_{con}$ we have $V = n$ and $s'' = s$.

We need to show $\langle \ell, s \rangle \Downarrow \langle n, s \rangle$ but this is exactly given by $\Downarrow_{loc}$, thus this holds.

(b).2 **Binary operations final step**

$$\langle n_1 \ op \ n_2, s \rangle \rightarrow \langle n, s \rangle \text{ with } n = n_1 \ op \ n_2$$

Assume $\langle n, s \rangle \Downarrow \langle V, s'' \rangle$. Since n is an integer final value, by $\Downarrow_{loc}$ we have $V = n$ and $s'' = s$.

By $(\Downarrow_{op})$, knowing that values evaluate themselves: $\langle n_1, s \rangle \Downarrow n_1$ and $\langle n_2, s \rangle \Downarrow n_2$, hence

$$\langle n_1 \ op \ n_2, s \rangle \Downarrow \langle n, s \rangle \text{ with } n = n_1 \ op \ n_2$$

(b).3 **Assignment (final step)**

$$\langle \ell := n, s \rangle \rightarrow \langle skip, s[\ell \mapsto n] \rangle$$

Assume $\langle skip, s[\ell \mapsto n] \rangle \Downarrow \langle V, s'' \rangle$, then we have $V = skip$ and $s'' = s[\ell \mapsto n]$.

By $(\Downarrow_{set})$, knowing that values evaluate themselves:

$$\langle \ell := n, s \rangle \Downarrow \langle skip, s[\ell \mapsto n] \rangle$$

(b).4 **If-true**

$$\langle \text{if } \text{true} \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle C_1, s \rangle$$

Assume $\langle C_1, s \rangle \Downarrow \langle V, s'' \rangle$. We know that $\langle \text{true}, s \rangle \Downarrow \langle \text{true}, s \rangle$ by $(\Downarrow_{con})$ so we can apply $(\Downarrow_{if1})$, hence

$$\langle \text{if } \text{true} \text{ then } C_1 \text{ else } C_2, s \rangle \Downarrow \langle V, s'' \rangle$$

(b).5 **If-false**

$$\langle \text{if } \text{false} \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle C_2, s \rangle$$

Assume $\langle C_2, s \rangle \Downarrow \langle V, s'' \rangle$. We know that $\langle \text{false}, s \rangle \Downarrow \langle \text{false}, s \rangle$ by $(\Downarrow_{con})$ so we can apply $(\Downarrow_{if2})$, hence

$$\langle \text{if } \text{false} \text{ then } C_1 \text{ else } C_2, s \rangle \Downarrow \langle V, s'' \rangle$$

(b).6 **Binary operations**

$$\frac{\langle E_1, s \rangle \rightarrow \langle E'_1, s' \rangle}{\langle E_1 \text{ op } E_2, s \rangle \rightarrow \langle E'_1 \text{ op } E_2, s' \rangle}$$

Assume $\langle E'_1 \text{ op } E_2, s' \rangle \Downarrow \langle n, s'' \rangle$, we know that $\langle E'_1, s \rangle \Downarrow \langle n_1, s' \rangle$ and $\langle E_2, s' \rangle \Downarrow \langle n_2, s'' \rangle$ By Inductive Hypothesis (IH): $\langle E_1, s \rangle \rightarrow \langle E'_1, s' \rangle$ and $\langle E'_1, s \rangle \Downarrow \langle n_1, s' \rangle$ implies $\langle E_1, s \rangle \Downarrow \langle n_1, s' \rangle$. Hence by $(\Downarrow_{op})$

$$\langle E_1 \text{ op } E_2, s \rangle \Downarrow \langle n, s'' \rangle$$

(b).7 **Assignment**

$$\frac{\langle E, s \rangle \rightarrow \langle E', s' \rangle}{\langle \ell := E, s \rangle \rightarrow \langle \ell := E', s' \rangle}$$

Assume $\langle \ell := E', s' \rangle \Downarrow \langle \text{skip}, s'[\ell \mapsto n] \rangle$, hence we can deduce $\langle E', s \rangle \Downarrow \langle n, s' \rangle$. By IH on $\langle \ell := E, s \rangle \rightarrow \langle E', s' \rangle$ we have that $\langle E, s \rangle \Downarrow \langle n, s' \rangle$. Now we can apply $(\Downarrow_{set})$ and obtain

$$\langle \ell := E, s \rangle \Downarrow \langle \text{skip}, s'[\ell \mapsto n] \rangle$$

(b).8 **If-Then-Else**

$$\frac{\langle B, s \rangle \rightarrow \langle B', s' \rangle}{\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle \rightarrow \langle \text{if } B' \text{ then } C_1 \text{ else } C_2, s' \rangle}$$

Assume $\langle \text{if } B' \text{ then } C_1 \text{ else } C_2, s' \rangle \Downarrow \langle V, s'' \rangle$ By IH on $\langle B, s \rangle \rightarrow \langle B', s' \rangle$ we know that if $\langle B', s' \rangle \Downarrow \langle b, s'' \rangle$ with $b \in \mathbb{B}$ then $\langle B, s \rangle \Downarrow \langle b, s'' \rangle$, hence this never change the evaluation of the boolean condition.

$$\langle \text{if } B \text{ then } C_1 \text{ else } C_2, s \rangle \Downarrow \langle V, s'' \rangle$$

(b).9 **Sequence**

$$\frac{\langle C_1, s \rangle \rightarrow \langle C'_1, s' \rangle}{\langle C_1; C_2, s \rangle \rightarrow \langle C'_1; C_2, s' \rangle}$$

Assume $\langle C'_1; C_2, s' \rangle \Downarrow \langle \text{skip}, s''' \rangle$. We deduce $\langle C'_1, s' \rangle \Downarrow \langle \text{skip}, s' \rangle$ and $\langle C_2, s'' \rangle \Downarrow \langle \text{skip}, s''' \rangle$. By IH, since $\langle C_1, s \rangle \rightarrow \langle C'_1, s' \rangle$ we have $\langle C_1, s \rangle \Downarrow \langle \text{skip}, s' \rangle$. Hence by $(\Downarrow_{seq})$ we obtain:

$$\langle C_1; C_2, s \rangle \Downarrow \langle \text{skip}, s''' \rangle$$

□

Exercise 4.3.1. Prove the LC semantic equivalences:

- (a) $C; \mathbf{skip} \cong C \cong \mathbf{skip}; C$
- (b) $(C; C'); C'' \cong C; (C'; C'')$
- (c) $\mathbf{if\ true\ then\ } C \mathbf{\ else\ } C' \cong C$
- (d) $\mathbf{if\ false\ then\ } C \mathbf{\ else\ } C' \cong C'$
- (e) $\mathbf{while\ } B \mathbf{\ do\ } C \cong \mathbf{if\ } B \mathbf{\ then\ } C; (\mathbf{while\ } B \mathbf{\ do\ } C) \mathbf{\ else\ skip}$
- (f) $\ell := n; \ell' := n' \cong \begin{cases} \ell' := n'; \ell := n & \text{if } \ell \neq \ell' \\ \ell := n' & \text{if } \ell = \ell' \end{cases}$

Solution.

- (a) $C; \mathbf{skip} \cong C \cong \mathbf{skip}; C$
 - (a).1 $C; \mathbf{skip} \cong C$
 - (a).1.1 ($\implies$): Suppose $\langle C; \mathbf{skip}, s \rangle \Downarrow s_1$ implies the existence of s' such that $\langle C, s \rangle \Downarrow s'$ and $\langle \mathbf{skip}, s' \rangle \Downarrow s_1$. By $(\Downarrow_{\mathbf{skip}})$ we have $s' = s_1$, hence $\langle C, s \rangle \Downarrow s_1$.
 - (a).1.2 ($\impliedby$): Suppose $\langle C, s \rangle \Downarrow s_1$. Since $\langle \mathbf{skip}, s \rangle \Downarrow s$ is always true by $(\Downarrow_{\mathbf{skip}})$ it's easy to obtain with $(\Downarrow_{\text{seq}})$ that $\langle C; \mathbf{skip}, s \rangle \Downarrow s_1$
 - (a).2 $\mathbf{skip}; C \cong C$
 - (a).2.1 ($\implies$): Suppose $\langle \mathbf{skip}; C, s \rangle \Downarrow s_1$. By $(\Downarrow_{\text{seq}})$ there must be a state s' such that $\langle \mathbf{skip}, s \rangle \Downarrow s'$ and $\langle C, s' \rangle \Downarrow s_1$.
By $(\Downarrow_{\mathbf{skip}})$ we have that $s' = s$, replacing we obtain $\langle C, s \rangle \Downarrow s_1$, q.e.d.
 - (a).2.2 ($\impliedby$): Suppose $\langle C, s \rangle \Downarrow s_1$. Since $\langle \mathbf{skip}, s \rangle \Downarrow s$ is always true by $(\Downarrow_{\mathbf{skip}})$ it's easy to obtain with $(\Downarrow_{\text{seq}})$ that $\langle \mathbf{skip}; C, s \rangle \Downarrow s_1$
- (b) $(C; C'); C'' \cong C; (C'; C'')$
 - (b).1 ($\implies$): Suppose that $\langle (C; C'); C'', s \rangle \Downarrow s_1$, for this to happen, by $(\Downarrow_{\text{seq}})$, there exists s'' such that $\langle (C; C'), s \rangle \Downarrow s''$ and $\langle C'', s'' \rangle \Downarrow s_1$.
Since $\langle (C; C'), s \rangle \Downarrow s''$ there is another intermediate state s' such that $\langle C, s \rangle \Downarrow s'$ and $\langle C', s' \rangle \Downarrow s''$.
Applying $(\Downarrow_{\text{seq}})$ on $(C'; C'')$ from state s' , we obtain $\langle C'; C'', s' \rangle \Downarrow s_1$. Then we can execute $\langle C, s \rangle \Downarrow s'$ before again by $(\Downarrow_{\text{seq}})$ we conclude $\langle C; (C'; C''), s \rangle \Downarrow s_1$, q.e.d.
 - (b).2 ($\impliedby$): Suppose that $\langle C; (C'; C''), s \rangle \Downarrow s_1$, for this to happen, by $(\Downarrow_{\text{seq}})$, there exists s' such that $\langle C, s \rangle \Downarrow s'$ and $\langle (C'; C''), s' \rangle \Downarrow s_1$.
Since $\langle (C'; C''), s' \rangle \Downarrow s_1$ there is another intermediate state s'' such that $\langle C', s' \rangle \Downarrow s''$ and $\langle C'', s'' \rangle \Downarrow s_1$.
Applying $(\Downarrow_{\text{seq}})$ we conclude $\langle (C; C'); C'', s \rangle \Downarrow s_1$, q.e.d.
- (c) $\mathbf{if\ true\ then\ } C \mathbf{\ else\ } C' \cong C$

Suppose $\langle \mathbf{if\ true\ then\ } C \mathbf{\ else\ } C', s \rangle \Downarrow s_1$. Since the boolean expression is always evaluated as *true*, the only applicable rule is $(\Downarrow_{if1})$ which execute the 'then' branch without changing the state, hence we obtain $\langle C, s \rangle \Downarrow s_1$
- (d) $\mathbf{if\ false\ then\ } C \mathbf{\ else\ } C' \cong C'$

Suppose $\langle \mathbf{if\ true\ then\ } C \mathbf{\ else\ } C', s \rangle \Downarrow s_1$. Specularly to the previous one, the boolean expression is always evaluated as *false*, the only applicable rule is $(\Downarrow_{if2})$ which execute the 'else' branch without changing the state, hence we obtain $\langle C', s \rangle \Downarrow s_1$

(e) **while** B **do** $C \cong \text{if } B \text{ then } C; (\text{while } B \text{ do } C) \text{ else skip}$

We proceed by cases on the value of the boolean guard B :

(e).1 B **is evaluated to** *false*: the loop terminates immediately in state s ($\langle \text{while } B \text{ do } C, s \rangle \Downarrow \langle \text{skip}, s \rangle$), by ($\Downarrow_{wh2}$). The corresponding *if* statement evaluates the guard to *false*, executes the *else* branch (which is *skip*), and also terminates in state s ($\langle \text{if } B \text{ then } C; (\text{while } B \text{ do } C) \text{ else skip}, s \rangle \Downarrow \langle \text{skip}, s \rangle$), by ($\Downarrow_{if2}$).

(e).2 B **is evaluated to** *true*: the loop executes the body C to a state s' , then execute the entire loop again, i.e. ($\langle \text{while } B \text{ do } C, s \rangle \Downarrow \langle \text{skip}, s'' \rangle$ if and only if $\langle C, s \rangle \Downarrow s'$ and $\langle \text{while } B \text{ do } C, s' \rangle \Downarrow \langle \text{skip}, s'' \rangle$).

On the other hand the *if* statement evaluates the guard to *true* executing command C and the entire while loop, producing the exact same sequence of states.

(f) $\ell := n; \ell' := n' \cong \begin{cases} \ell' := n'; \ell := n & \text{if } \ell \neq \ell' \\ \ell := n' & \text{if } \ell = \ell' \end{cases}$

We proceed by cases:

(f).1 **Case** $\ell \neq \ell'$: the two assignments act on distinct memory locations. Since the order in which independent locations are written does not affect the final contents of each (memory operations are "side-effect free" with respect to other unaffected locations), the order can be reversed without changing the final state of the memory.

(f).2 **Case** $\ell = \ell'$: if the locations match, the second assignment $\ell := n'$ completely overwrites the value previously written by the first $\ell := n$. Therefore, the net effect on the final memory state is identical to that of the single command $\ell := n'$, making the first assignment redundant.

All equivalences are validated by comparing the partial functions (initial state $\leftrightarrow$ final state) generated by the inference rules of LC semantics. If the derivations lead to the same final state for every possible input, the commands are semantically indistinguishable. $\square$

Exercise 5.5.2. What is wrong with the following suggestion?

“ The rule

$$(\Downarrow_{\text{letrec}}) \frac{M_2[(\text{letrec } x = M_1 \text{ in } M_1)/x] \Downarrow V,}{\text{letrec } x = M_1 \text{ in } M_2 \Downarrow V}$$

can be simplified to

$$\frac{M_2[(\text{letrec } x = M_1 \text{ in } x)/x] \Downarrow V}{\text{letrec } x = M_1 \text{ in } M_2 \Downarrow V}$$

because in the body of the **letrec**-expression, x id defines to be M_1 so we can use **letrec** $x = M_1$ **in** x instead of **letrec** $x = M_1$ **in** M_1 .”

Hint: consider **letrec** $x = 0$ **in** x .

Solution. The fundamental error in the proposed simplification of the rule $(\Downarrow_{\text{letrec}})$ lies in the fact that it changes the nature of the fixed point of recursion, transforming computations that should terminate into an infinite divergence (non-termination). In short, while the original rule allows one to “unroll” the recursive definition in order to access the body of the function or expression (M_1), the simplified rule replaces the variable with a term that refers back to itself circularly, without ever making progress toward evaluating M_1 .

1. Detailed Analysis with the Example: **letrec** $x = 0$ **in** 0

To show why the suggestion is incorrect, we apply both rules to the expression proposed: **letrec** $x = 0$ **in** 0.

In this case, we have:

- $M_1 = 0$ (by definition)
- $M_2 = x$ (by the body of the expression)

1.1 Evaluation Using the Original Rule

$$(\Downarrow_{\text{letrec}}) \frac{M_2[(\text{letrec } x = M_1 \text{ in } M_1)/x] \Downarrow V,}{\text{letrec } x = M_1 \text{ in } M_2 \Downarrow V}$$

- 1.1.1 We must evaluate the body M_2 (which is x), replacing every occurrence of x with the term:

$$\text{letrec } x = 0 \text{ in } 0$$

- 1.1.2 The premise expression becomes:

$$x[(\text{letrec } x = 0 \text{ in } 0)/x]$$

which is

$$\text{letrec } x = 0 \text{ in } 0$$

- 1.1.3 To evaluate **letrec** $x = 0$ **in** 0, we apply the rule again. The new body is 0. We substitute x in the body, but since x does not occur in 0, the substitution has no effect.

- 1.1.4 We then evaluate the final premise:

$$0 \Downarrow 0$$

which holds by the value axiom.

- 1.1.5 **Result:** the expression terminates correctly and returns a value.

1.2 Evaluation Using the Simplified (Incorrect) Rule

$$\frac{M_2[(\text{letrec } x = M_1 \text{ in } x)/x] \Downarrow V}{\text{letrec } x = M_1 \text{ in } M_2 \Downarrow V}$$

1.2.1 We must evaluate the body M_2 (i.e. x), replacing every occurrence with:

letrec $x = 0$ **in** x

1.2.2 The premise expression becomes:

$x[(\text{letrec } x = 0 \text{ in } 0)/x]$

which is

letrec $x = 0$ **in** 0

1.2.3 To evaluate this premise, we apply the same rule again.

1.2.4 This forces us to evaluate once more

letrec $x = 0$ **in** 0

as the premise of the premise.

1.2.5 This generates an infinite chain of identical derivations that never reaches the constant 0.

1.2.6 **Result:** the expression diverges and never returns a value.

2. Why the Simplification Fails

The failure comes from an error on how local recursive definitions work:

- **Syntactic Identity vs. Evaluation:** the term **letrec** $x = M_1$ **in** x is syntactically equivalent to the entire expression we are trying to evaluate. Using it as a replacement for x means telling the semantic machine: “To know what x is, look at the entire program again.” This creates an immediate loop.
- **Loss of Access to the Definition:** In a **letrec**, x is the name of the recursion, but its value is derived from M_1 . Original rule replaces x with a term that points to M_1 :

letrec $x = M_1$ **in** M_1

ensuring that evaluation eventually moves from the variable to the content of the definition.

- **Lack of Progress:** A valid inference rule must allow computation to progress toward a normal form or value. The proposed simplification violates this principle because the premise becomes identical to the conclusion, preventing any reduction.

3. Conclusion

The suggestion is wrong because it confuses the *reference* of the recursion (x) with the body of the recursion (M_1). By simplifying the rule as proposed, we obtain a language in which no recursive definition can ever be resolved, because every attempt to evaluate the recursive variable results in an infinite self-loop. This makes the **letrec** construct completely useless for computation.

□

Exercise 5.5.3. *Prove*

(Subject reduction) *If $M : \tau$ and $M \Downarrow_n V$ then $V : \tau$ by Rule Induction. Show that the property $\Phi(M, V)$ defined by*

$$M \Downarrow_n V \& \forall \tau. M : \tau \implies V : \tau$$

is closed under the axioms and rules inductively defining $\Downarrow_n$.

(For closure under rule $(\Downarrow_{cbn})$ you will need

(Lemma 5.3.2) *If $\Gamma \vdash M : \tau$ and $\Gamma[x \mapsto \tau] \vdash M' : \tau'$ with $x \notin \text{dom}(\Gamma)$, then $\Gamma \vdash M'[M/x] : \tau'$.*

If you are really keen, try proving that, by induction on the structure of M' .)

Solution.

We want to show that the property $\Phi(M, V) \equiv \forall \tau. (M : \tau \implies V : \tau)$ is closed under the axioms and rules inductively defining $M \Downarrow_n V$.

1. Values:

$$(\Downarrow_{val}) \quad V \Downarrow V$$

where V is an integer or a boolean value or a lambda function.

Suppose $M = V$ and $M : \tau$, then trivially $V : \tau$ since the term does not change.

The property $\Phi(V, V)$ holds.

2. Binary operations:

$$(\Downarrow_{op}) \quad \frac{M_1 \Downarrow_{n_1} M_2 \Downarrow_{n_2}}{M_1 \text{ op } M_2 \Downarrow_n} \text{ with } n = n_1 \text{ op } n_2$$

Suppose $M_1 \text{ op } M_2 : \tau$ then $M_1 : \tau$ and $M_2 : \tau$.

Hence by inductive hypothesis (IH) $n_1 : \tau$ and $n_2 : \tau$, thus $n = n_1 \text{ op } n_2 : \tau$ by definition of op . Therefore $V : \tau$ holds.

3. Memory lookup:

$$(\Downarrow_!) \quad \frac{M \Downarrow \ell}{!M_1 \Downarrow n} \text{ if } s(\ell) = n$$

Assume $!M : \tau$, typing implies $M : loc$

By inductive hypothesis (IH) on $M \Downarrow \ell$:

$$\ell : loc$$

Store typing guarantees $s(\ell) = n$ has type τ . Hence $n : \tau$.

So $\Phi(!M, n)$ holds.

4. Assignment:

$$(\Downarrow_{:=}) \quad \frac{M_1 \Downarrow \ell \quad M_2 \Downarrow n}{M_1 := M_2 \Downarrow \langle skip, s[\ell \mapsto n] \rangle}$$

Assume $M_1 := M_2 : \tau$.

Typing requires:

$$\tau = comm \quad M_1 : loc \quad M_2 : int$$

By IH:

$$\ell : loc \quad n : int$$

The evaluation result is $skip$, hence $\Phi(M_1 := M_2, skip)$ holds. The evaluation result is $skip$, typing rules give $skip : comm$. Thus $skip : \tau$.

So $\Phi(M_1 := M_2, skip)$ holds.

5. **Sequence:**

$$(\Downarrow_{seq}) \quad \frac{M_1 \Downarrow skip \quad M_2 \Downarrow skip}{M_1; M_2 \Downarrow skip}$$

Assume $M_1; M_2 : \tau$. Typing gives

$$\tau : comm \quad M_1 : comm \quad M_2 : comm$$

By IH $skip : comm$, this satisfies $skip : \tau$.

So $\Phi(M_1; M_2, skip)$ holds.

6. **If:**

6.1 **If-True:**

$$(\Downarrow_{if1}) \quad \frac{M_1 \Downarrow true \quad M_2 \Downarrow V}{\text{if } M_1 \text{ then } M_2 \text{ else } M_3 \Downarrow V}$$

6.2 **If-False:**

$$(\Downarrow_{if2}) \quad \frac{M_1 \Downarrow false \quad M_3 \Downarrow V}{\text{if } M_1 \text{ then } M_2 \text{ else } M_3 \Downarrow V}$$

Assume $\text{if } M_1 \text{ then } M_2 \text{ else } M_3 : \tau$, typing gives:

$$M_1 : bool \quad M_2 : \tau \quad M_3 : \tau$$

By IH: $V : \tau$ and $true : bool$ ($false : bool$ for [6.2]).

Hence preservation holds.

7. **While:**

7.1 **While-True:**

$$(\Downarrow_{wh1}) \quad \frac{M_1 \Downarrow true \quad M_2 \Downarrow V \quad \text{while } M_1 \text{ do } M_2 \Downarrow skip}{\text{while } M_1 \text{ do } M_2 \Downarrow skip}$$

7.2 **While-False:**

$$(\Downarrow_{wh2}) \quad \frac{M_1 \Downarrow false}{\text{while } M_1 \text{ do } M_2 \Downarrow skip}$$

Typing gives

$$M_1 : bool \quad M_2 : comm \quad \text{while } M_1 \text{ do } M_2 : comm$$

By IH:

$$[7.1] \quad M_1 : bool \quad M_2 : comm \quad \text{while } M_1 \text{ do } M_2 : comm$$

$$[7.2] \quad M_1 : bool \quad M_2 : comm$$

The result is $skip : comm$.

Hence the preservation holds.

8. **Call-by-Name:**

$$(\Downarrow_{cbn}) \quad \frac{M_1 \Downarrow \lambda x.M'_1 \quad M'_1[M_2/x] \Downarrow V}{M_1 M_2 \Downarrow V}$$

Assume $M_1 M_2 : \tau$. Typing rules for application implies:

$$M_1 : \omega \rightarrow \tau \quad M_2 : \omega$$

By inductive hypothesis on the first premise:

$$\lambda x.M'_1 : \omega \rightarrow \tau$$

Thus typing of abstraction gives:

$$x : \omega \vdash M'_1 : \tau$$

Now by (**Lemma 5.3.2**):

$$M_2 : \omega \implies M'_1[M_2/x] : \tau$$

By IH on the second premise: $V : \tau$. So the property holds.

9. Call-by-Value:

$$(\Downarrow_{cbv}) \quad \frac{M_1 \Downarrow \lambda x.M'_1 \quad M_2 \Downarrow V_2 \quad M'_1[V_2/x] \Downarrow V}{M_1 M_2 \Downarrow V}$$

Typing gives:

$$M_1 : \omega \rightarrow \tau \quad M_2 : \omega$$

By inductive hypothesis: $V_2 : \omega$. By IH on the final premise: $V : \tau$. So the property holds.

We have shown closure of $\Phi(M, V)$ under every evaluation rule. Therefore:

$$M \Downarrow_n V \& \forall \tau. M : \tau \implies V : \tau$$

□

Exercise Call-by-name equivalence. Consider the following definition of small-step call-by-name reduction $\rightarrow_n$:

$$\frac{}{app(lam(x.M), N) \rightarrow_n M[x := N]} \quad \frac{M \rightarrow_n M'}{app(M, N) \rightarrow_n app(M', N)}$$

Show that $M \Downarrow_n V$ if and only if $M \rightarrow_n^* V$.

Solution. We prove both implications.

1. ($\Rightarrow$) If $M \Downarrow V$ then $M \rightarrow_n^* V$

We proceed by induction on the derivation of $M \Downarrow_n V$. We need to show that if a term evaluates to a value in a single "big step," there exists a sequence of atomic transitions (reflexive-transitive closure $\rightarrow_n^*$) that achieves the same result.

1.1 BASE CASE - AXIOM: Assume $lam(x.M) \Downarrow lam(x.M)$. By the definition of reflexive-transitive closure, every configuration is related to itself in zero steps $lam(x.M) \rightarrow_n^0 lam(x.M)$. Therefore, $lam(x.M) \rightarrow_n^* lam(x.M)$ is trivially true.

1.2 INDUCTIVE STEP - RULE: Assume $app(M_1, M_2) \Downarrow V$. By inductive hypothesis on the premises.

1.2.1 on $M_1 \Downarrow lam(x.M)$ we have $M_1 \rightarrow_n^* lam(x.M)$;

1.2.2 on $M[x := M_2] \Downarrow V$ we have $M[x := M_2] \rightarrow_n^* V$.

From the second transition rule given (congruence), we know that if M_1 can take steps, then the entire application can progress: $app(M_1, M_2) \rightarrow_n^* app(lam(x.M), M_2)$. From the first transition rule (β -reduction): $app(lam(x.M), M_2) \rightarrow_n M[x := M_2]$.

Joining the sequences by transitivity:

$$app(M_1, M_2) \rightarrow_n^* app(lam(x.M), M_2) \rightarrow_n M[x := M_2] \rightarrow_n^* V$$

We conclude that

$$app(M_1, M_2) \rightarrow_n^* V$$

2. ($\Leftarrow$) If $M \rightarrow_n^* V$ then $M \Downarrow V$ We proceed by induction on the length of the reduction sequence $M \rightarrow_n^k V$.

2.1 BASE CASE ($k = 0$): Assume $M \rightarrow_n^0 V$, then $M = V$. Since V is a value (lambda abstraction), by the axiom ($\Downarrow_{val}$) we have $V \Downarrow V$.

2.2 INDUCTIVE STEP ($k = n + 1$): Assume $M \rightarrow_n M' \rightarrow_n^n V$. By inductive hypothesis we know $M' \Downarrow V$. To conclude $M \Downarrow V$ we show (**Lemma**): if $M \rightarrow_n M'$ and $M' \Downarrow V$ then $M \Downarrow V$.

We show the (**Lemma**) by cases on $M \rightarrow M'$:

2.2.1 (β -REDUCTION): $app(lam(x.M), N) \rightarrow_n M[x := N]$

Assume $M[x := N] \Downarrow V$.

Since we know by ($\Downarrow_{val}$) that $lam(x.M) \Downarrow lam(x.M)$ we obtain, by Big-Step rule for application, $app(lam(x.M), N) \Downarrow V$

2.2.2 (STRUCTURAL RULE): $app(M, N) \rightarrow_n app(M', N)$ with $M \rightarrow_n M'$

Assume $app(M', N) \Downarrow V$, by inversion there must be M and P such that $M' \Downarrow lam(x.P)$ and $P[x := N] \Downarrow V$. By inductive hypothesis on $M \rightarrow_n M'$ and $M' \Downarrow lam(x.P)$ we obtain $M \Downarrow lam(x.P)$.

Now we can use Big-Step rule for application and conclude $app(M, N) \Downarrow V$.

Equivalence is demonstrated because every Big-Step derivation tree can be linearized into a sequence of Small-Step transitions and each Small-Step transition step preserves the final "meaning" of the term defined by the Big-Step relation. □